

3.6 break and continue

[This section corresponds to K&R Sec. 3.7]

Sometimes, due to an exceptional condition, you need to jump out of a loop early, that is, before the main controlling expression of the loop causes it to terminate normally. Other times, in an elaborate loop, you may want to jump back to the top of the loop (to test the controlling expression again, and perhaps begin a new trip through the loop) without playing out all the steps of the current loop. The `break` and `continue` statements allow you to do these two things. (They are, in fact, essentially restricted forms of `goto`.)

To put everything we've seen in this chapter together, as well as demonstrate the use of the `break` statement, here is a program for printing prime numbers between 1 and 100:

```
#include <stdio.h>
#include <math.h>

main()
{
    int i, j;

    printf("%d\n", 2);

    for(i = 3; i <= 100; i = i + 1)
    {
        for(j = 2; j < i; j = j + 1)
        {
            if(i % j == 0)
                break;
            if(j > sqrt(i))
            {
                printf("%d\n", i);
                break;
            }
        }
    }

    return 0;
}
```

The outer loop steps the variable `i` through the numbers from 3 to 100; the code tests to see if each number has any divisors other than 1 and itself. The trial divisor `j` loops from 2 up to `i`. `j` is a divisor of `i` if the remainder of `i` divided by `j` is 0, so the code uses C's ``remainder" or ``modulus" operator `%` to make this test. (Remember that `i % j` gives the remainder when `i` is divided by `j`.)

If the program finds a divisor, it uses `break` to break out of the inner loop, without printing anything. But if it notices that `j` has risen higher than the square root of `i`, without its having found any divisors, then `i` must not have any divisors, so `i` is prime, and its value is printed. (Once we've determined that `i` is prime by noticing that `j > sqrt(i)`, there's no need to try the other trial divisors, so we use a second `break` statement to break out of the loop in that case, too.)

The simple algorithm and implementation we used here (like many simple prime number algorithms) does not work for 2, the only even prime number, so the program ``cheats" and prints out 2 no matter what, before going on to test the numbers from 3 to 100.

Many improvements to this simple program are of course possible; you might experiment with it. (Did you notice that the ``test" expression of the inner loop `for(j = 2; j < i; j = j + 1)` is in a

sense unnecessary, because the loop always terminates early due to one of the two `break` statements?)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail](#) [feedback](#)